Section 5: Hash Maps and Hash Sets

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

SLIDE 40: Section Intro, Hash Maps and Hash Sets

SLIDE 41: How Hashing Works and Operation Costs

SLIDE 42: The Two Sum Pattern

SLIDE 43: Worked Example: Two Sum Step-by-Step

SLIDE 44: Frequency Counting and Grouping

SLIDE 45: Deduplication, Existence, and Lookup Optimization

SLIDE 45B: Worked Example: Longest Consecutive Sequence

SLIDE 46: Worked Example: Contains Duplicate II

SLIDE 47: Wrong vs Right: Hash Map Mistakes

SLIDE 48: How to Present Hash Map Solutions in an Interview

SLIDE 49: Interview Practice: Hash Maps

SLIDE 50: Section Cheat Sheet

SLIDE 40: Section Intro, Hash Maps and Hash Sets

Section 5: Hash Maps and Hash Sets

Hash maps and hash sets are the single most important tool for optimizing brute force solutions in interviews. Whenever your brute force uses a nested loop where the inner loop searches for a matching element, a hash map can almost always replace that inner scan with an O(1) lookup, dropping the total complexity from O(n^2) to O(n).

Recognition Signals

You should reach for a hash map or hash set when you see any of these in a problem: need to check if an element exists in a collection, need to count how many times each element appears, need to associate data with a key for fast retrieval, or your brute force has nested loops where the inner loop is searching for a match.

What We'll Cover

1. How hashing works and operation costs
2. The Two Sum pattern and its generalizations
3. Frequency counting and grouping
4. Deduplication, existence checking, and lookup optimization
5. Contains Duplicate II as a combined worked example

Connection to Other Sections

Character frequency maps from Section 4 are hash maps applied to characters. The same int[26] technique is a specialized hash map with a perfect hash function. Two Sum can also be solved with two pointers on sorted input (Section 6), but the hash map approach works on unsorted input without modifying the array. Sliding window problems in Section 7 often use a hash map or hash set internally to track window state.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 41: How Hashing Works and Operation Costs

A hash map stores key-value pairs. Internally, it computes a hash code from the key, maps that to a bucket index, and stores the entry in that bucket. When two different keys produce the same bucket index, a collision occurs. Java's HashMap handles collisions by chaining entries in the same bucket (using a linked list, upgraded to a balanced tree at 8+ entries per bucket).

The practical takeaway: average-case operations are O(1). Worst case is O(n) if every key collides into the same bucket, but this almost never happens with a reasonable hash function.

Operation Costs

Operation	HashMap	HashSet	TreeMap
put / add	O(1) avg	O(1) avg	O(log n)
get / contains	O(1) avg	O(1) avg	O(log n)
remove	O(1) avg	O(1) avg	O(log n)
containsValue	O(n)	N/A	O(n)
Iterate all entries	O(n)	O(n)	O(n)
Ordering guarantee	None	None	Sorted by key

Map vs Set: Which One Do I Need?

Question	Answer	Use
Do I need to associate a value with each key?	Yes	HashMap
Do I only need to know if an element exists?	Yes	HashSet
Do I need elements in sorted order?	Yes	TreeMap / TreeSet
Do I need insertion order preserved?	Yes	LinkedHashMap / LinkedHashSet

Java Specifics

HashMap allows one null key and multiple null values. Keys must be effectively immutable. If you modify an object after using it as a key, the map cannot find it. Use getOrDefault(key, defaultValue) for clean null handling instead of checking containsKey first.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 42: The Two Sum Pattern

Two Sum is the most frequently asked interview problem across all major companies. More importantly, the hash map technique it uses generalizes to dozens of other problems. Learning Two Sum is not about one problem. It is about learning the pattern of replacing an O(n) inner scan with an O(1) hash map lookup.

Brute Force: O(n^2) Time, O(1) Space

```
public int[] twoSum(int[] nums, int target) {
    for (int i = 0; i < nums.length; i++) {
        for (int j = i + 1; j < nums.length; j++) {
            if (nums[i] + nums[j] == target) {
                return new int[]{i, j};
            }
        }
    }
    return new int[]{};
}
```

For each element, the inner loop scans the remaining array looking for a complement. That scan is where the time is wasted.

Optimal: O(n) Time, O(n) Space

```java
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int complement = target - nums[i];
        if (map.containsKey(complement)) {
            return new int[]{map.get(complement), i};
        }
        map.put(nums[i], i);
    }
    return new int[]{};
}
```

The Core Insight

The brute force asks “does my complement exist?” by scanning the entire array. The hash map answers the same question with a single O(1) lookup. Every time your brute force has a nested loop where the inner loop is searching for a match, this is the transformation to apply.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 43: Worked Example: Two Sum Step-by-Step

Trace Through [2, 7, 11, 15] with Target 9

```
i=0: nums[0]=2, complement=9-2=7
     map={}, 7 not found
     Store: map={2:0}

i=1: nums[1]=7, complement=9-7=2
     map={2:0}, 2 found at index 0
     Return [0, 1]
```

Why One Pass Works

When we process index 1 (value 7), its complement (value 2) was already stored from index 0. We never need a second pass because by the time we reach any element, all previous elements are already in the map. If a valid pair exists, the later element will always find its complement already stored.

Two Sum Variants

Variant	Approach	Time	Space	Key Difference
Two Sum (return indices)	HashMap value-to-index	O(n)	O(n)	Need original indices, cannot sort
Two Sum (return boolean)	HashSet of complements	O(n)	O(n)	No index tracking needed, HashSet suffices
Two Sum on sorted input	Two pointers (Section 6)	O(n)	O(1)	Sorted order enables pointer movement logic
Three Sum	Sort + fix one + two pointers	O(n^2)	O(1)	Reduces to Two Sum for each fixed element
Two Sum count all pairs	HashMap frequency counting	O(n)	O(n)	Count complement occurrences, handle duplicates

The Generalization

Every nested loop where the inner loop searches for a match can likely be replaced with a hash map lookup. This is not just a Two Sum trick. It is the fundamental hash map optimization that applies across dozens of interview problems.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 44: Frequency Counting and Grouping

Frequency Counting

The standard Java idiom for counting occurrences:

```
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}
```

Every time you need to count how many times each element appears, this is the template. The getOrDefault call handles the case where the key is not yet in the map, avoiding a separate containsKey check.

Worked Inline: First Non-Repeating Character (LeetCode 387)

Given a string, find the index of the first character that appears exactly once. Two-pass approach: first pass counts frequencies, second pass finds the first character with count 1.

```java
public int firstUniqChar(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    for (int i = 0; i < s.length(); i++) {
        if (freq.get(s.charAt(i)) == 1) {
            return i;
        }
    }
    return -1;
}
```

Time O(n), Space O(1) since the character set is bounded (at most 26 lowercase letters).

Common Frequency Problems

Problem	What You Count	Decision After Counting
First unique character	Character frequencies	First with count == 1
Most frequent element	Element frequencies	Max frequency
Elements appearing > n/k times	Element frequencies	Filter by threshold
Valid anagram	Character frequencies	All counts match
Top K frequent elements	Element frequencies	Sort or heap by count

Grouping Pattern

When you need to group elements by a computed property, use a HashMap where the key is the property and the value is a list.

```java
Map<String, List<String>> groups = new HashMap<>();
for (String word : words) {
    char[] chars = word.toCharArray();
    Arrays.sort(chars);
    String key = new String(chars);
    groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
}
```

This is the Group Anagrams pattern. The sorted character array is the grouping key because all anagrams produce the same sorted string. computeIfAbsent creates the list on first access, making the code cleaner than a manual containsKey check.

SLIDE 45: Deduplication, Existence, and Lookup Optimization

HashSet for Deduplication

To remove duplicates from a collection, add all elements to a HashSet. The set retains only unique values by definition.

```
Set<Integer> unique = new HashSet<>(Arrays.asList(1, 2, 3, 2, 1));
```

The set now contains {1, 2, 3}. Duplicates are discarded automatically.

HashSet for Existence Checking

ArrayList.contains() is O(n) per call. If you call it inside a loop, you get O(n^2). HashSet.contains() is O(1) per call. This is one of the most common optimizations in interview code.

```
List<Integer> list = Arrays.asList(1, 2, 3, 4, 5);
list.contains(3);

Set<Integer> set = new HashSet<>(list);
set.contains(3);
```

Both return true, but the set version is O(1) instead of O(n). In a loop processing n elements, this is the difference between O(n^2) and O(n).

The Lookup Optimization Generalization

This is the same insight as Two Sum, stated more broadly. Any time you have:

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (someCondition(arr[i], arr[j])) { ... }
    }
}
```

Ask yourself: “Can I preload arr[j] values into a hash set or hash map so the inner loop becomes a single O(1) lookup?” If yes, you drop from O(n^2) to O(n).

Idiomatic Duplicate Check

HashSet.add() returns false if the element already exists. You can use this as a one-line duplicate check:

```
if (!set.add(num)) {
    return true;
}
```

This is cleaner than calling contains() then add() separately. It performs both the check and the insertion in one operation.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 45B: Worked Example: Longest Consecutive Sequence

Problem: Longest Consecutive Sequence (LeetCode 128)

Given an unsorted integer array, find the length of the longest sequence of consecutive integers. The sequence does not need to be contiguous in the array. Must run in O(n) time.

Why Sorting Does Not Work Here

Sorting gives O(n log n), but the problem requires O(n). We need a different approach.

The Insight

Put all elements in a HashSet. A sequence starts at an element where element-1 is NOT in the set. From each sequence start, count how far the sequence extends by checking element+1, element+2, etc. in the set.

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int num : nums) {
        set.add(num);
    }
    int longest = 0;
    for (int num : set) {
        if (!set.contains(num - 1)) {
            int current = num;
            int streak = 1;
            while (set.contains(current + 1)) {
                current++;
                streak++;
```

```
            }
            longest = Math.max(longest, streak);
        }
    }
    return longest;
}
```

Trace Through [100, 4, 200, 1, 3, 2]

```
Set = {1, 2, 3, 4, 100, 200}

num=1: 0 not in set → sequence start
       1→2→3→4, streak=4, longest=4

num=2: 1 is in set → not a sequence start, skip

num=3: 2 is in set → not a sequence start, skip

num=4: 3 is in set → not a sequence start, skip

num=100: 99 not in set → sequence start
         streak=1, longest still 4

num=200: 199 not in set → sequence start
         streak=1, longest still 4

Result: 4
```

Why This is O(n) Despite the Nested Loop

The while loop looks like it could make this O(n^2), but each element is visited at most twice: once in the outer loop and at most once as part of some sequence's extension. The total work across all iterations of the inner while loop is at most n. This amortized argument is the same one that applies to sliding window (Section 7) and monotonic stack (Section 10).

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 46: Worked Example: Contains Duplicate II

Problem: Contains Duplicate II (LeetCode 219)

Given an integer array nums and an integer k, return true if any value appears at least twice within index distance k. Formally, return true if nums[i] == nums[j] and abs(i - j) <= k for some i != j.

Step 1: Understand

"I need to find if any value appears twice where the two occurrences are at most k positions apart. So it is not just 'does a duplicate exist' but 'does a duplicate exist within a window of size k.'"

Step 2: Identify the Pattern

"I need to look up whether I've seen this value recently, and I need to know where I saw it. 'Lookup with associated data' maps to HashMap storing value to most recent index."

Step 3: Plan

"Iterate through the array. For each element, check if it exists in the map and if the stored index is within k of the current index. Then update the map with the current index."

Solution 1: HashMap Approach. O(n) time, O(n) space.

```
public boolean containsNearbyDuplicate(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        if (map.containsKey(nums[i]) && i - map.get(nums[i]) <= k) {
            return true;
        }
        map.put(nums[i], i);
    }
    return false;
}
```

Solution 2: Sliding Window with HashSet. O(n) time, O(min(n, k)) space.

```
public boolean containsNearbyDuplicate(int[] nums, int k) {
    Set<Integer> window = new HashSet<>();
    for (int i = 0; i < nums.length; i++) {
        if (i > k) {
            window.remove(nums[i - k - 1]);
        }
        if (!window.add(nums[i])) {
            return true;
        }
    }
    return false;
}
```

Trace Through [1, 2, 3, 1] with k=3 (HashMap Approach)

```
i=0: nums[0]=1, map={}, not found. Store: map={1:0}
i=1: nums[1]=2, map={1:0}, not found. Store: map={1:0, 2:1}
i=2: nums[2]=3, not found. Store: map={1:0, 2:1, 3:2}
i=3: nums[3]=1, map has 1 at index 0. Distance: 3-0=3 <= 3. Return true.
```

This problem demonstrates both the HashMap pattern (lookup with associated data) and previews the sliding window concept from Section 7 (maintaining a fixed-size window of elements). Solution 2 explicitly maintains a window of size k using a HashSet, which is the same mental model you will use throughout Section 7.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 47: Wrong vs Right: Hash Map Mistakes

Wrong: Using list.contains() in a Loop

ArrayList.contains() is O(n). Inside a loop processing n elements, this produces O(n^2) total. If you only need existence checking, convert to a HashSet first for O(1) per lookup.

Wrong: HashMap When Only Existence is Needed

If you do not need to associate data with each key, use a HashSet. It is simpler and communicates your intent more clearly. The interviewer notices when you use the right tool.

Wrong: containsValue() in a Loop

containsValue() scans every entry in the map. It is O(n) per call. Inside a loop, this becomes O(n^2). If you need to check values frequently, build a reverse map (value-to-key) or a separate HashSet of values.

Wrong: Ignoring HashSet.add() Return Value

HashSet.add() returns false if the element already exists. Many freshers write:

```
if (set.contains(num)) {
    return true;
}
set.add(num);
```

This works but performs two hash lookups. The idiomatic version does it in one:

```
if (!set.add(num)) {
    return true;
```

```
}
```

Wrong vs Right Summary

Mistake	Cost	Fix
list.contains() in a loop	O(n) per call, O(n^2) total	Convert to HashSet first
HashMap for existence-only checks	Unnecessary complexity	Use HashSet
containsValue() in a loop	O(n) per call	Build reverse map or value set
Separate contains + add calls	Two hash lookups	Use !set.add() idiom
Not stating the space trade-off	Interviewer has to ask	State O(n) space cost unprompted
Using TreeMap when order is not needed	O(log n) instead of O(1)	HashMap unless sorted order is required

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 48: How to Present Hash Map Solutions in an Interview

How to Introduce the Optimization

Do not just say “I’ll use a hash map.” Explain the transformation from brute force:

“The brute force uses a nested loop to search for the complement. I can replace that inner scan with a hash map lookup, dropping from O(n^2) to O(n) at the cost of O(n) space.”

This sentence does three things: acknowledges the brute force, explains the mechanism of improvement, and states the trade-off. The interviewer now knows you understand why the hash map helps, not just that it does.

When to State the Trade-Off

Always state both time and space complexity. The interviewer expects you to acknowledge the space cost. “This is O(n) time and O(n) space. We’re trading memory for speed” is the statement that demonstrates engineering awareness. Omitting the space cost makes it look like you did not consider it.

How to Handle “Can You Do It Without Extra Space?”

This follow-up tests whether you know alternative approaches. Common responses:

Problem	With HashMap	Without Extra Space
Two Sum (indices)	O(n) time, O(n) space	Not possible without hash map if indices are needed and input is unsorted
Two Sum (boolean)	O(n) time, O(n) space	Sort + two pointers: O(n log n) time, O(1) space
Contains Duplicate	O(n) time, O(n) space	Sort first: O(n log n) time, O(1) space
Find pair with difference K	O(n) time, O(n) space	Sort + two pointers: O(n log n) time, O(1) space

The pattern: sorting is usually the O(1)-space alternative, but it costs O(n log n) time and may lose original index information. State this trade-off explicitly.

Interview Tip: If the interviewer asks for O(1) space on a problem where you need original indices (like Two Sum returning index pairs), it is appropriate to say: “The hash map is necessary here because sorting would change the indices. There is no O(n) time, O(1) space solution for this variant.” Knowing the limits of an approach is as valuable as knowing the approach itself.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 49: Interview Practice: Hash Maps

Practice Problem

“Given an array of integers, find the first element that appears exactly once.”

Expected Walkthrough

Step 1: Understand

“I’m given an array and I need to return the first element whose count is exactly 1. If no such element exists, return -1. I need to preserve order, so I need to find the first one, not just any one.”

Step 2: Identify the Pattern

“I need to count how many times each element appears, then find the first with count 1. This is a frequency counting problem, which maps to a HashMap.”

Step 3: Plan

“First pass: count frequencies using a HashMap. Second pass: iterate the original array and return the first element with frequency 1.”

Step 4: Implement

```java
public int firstUniqueElement(int[] nums) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int num : nums) {
        freq.put(num, freq.getOrDefault(num, 0) + 1);
    }
    for (int num : nums) {
        if (freq.get(num) == 1) {
            return num;
        }
    }
    return -1;
}
```

Step 5: Verify

"For [4, 5, 1, 2, 1, 4, 2]: frequencies are {4:2, 5:1, 1:2, 2:2}. Second pass checks 4 (count 2, skip), 5 (count 1, return 5). Correct."

Time O(n), Space O(n).

Why HashMap Over Sorting

Sorting would group duplicates together, but it destroys the original order. The problem asks for the "first" unique element, which depends on original position. HashMap preserves this because we iterate the original array in the second pass.

Common Fresher Mistakes

Mistake	Problem
Using nested loops to count each element	O(n^2) when HashMap gives O(n)
Iterating the map instead of the original array	HashMap does not guarantee insertion order; you lose "first" semantics
Using TreeMap instead of HashMap	O(n log n) for no benefit; sorted keys are irrelevant here
Returning the first key with count 1 from the map	Wrong answer; map iteration order is not array order

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 50: Section Cheat Sheet

Operation Complexity Comparison

Operation	HashMap	HashSet	TreeMap	TreeSet	ArrayList
Insert	O(1) avg	O(1) avg	O(log n)	O(log n)	O(1) amortized
Lookup	O(1) avg	O(1) avg	O(log n)	O(log n)	O(n)
Remove	O(1) avg	O(1) avg	O(log n)	O(log n)	O(n)
Ordered?	No	No	Yes	Yes	By index

Pattern Recognition Signals

Problem Signal	Pattern	Technique
“Find pair summing to target”	Two Sum	HashMap value-to-index
“Count occurrences of each element”	Frequency counting	HashMap with getOrDefault
“Group elements by property”	Grouping	HashMap property-to-list with computeIfAbsent
“Check if element exists in collection”	Existence check	HashSet
“Replace nested loop that searches for match”	Lookup optimization	Preload into HashMap or HashSet
“Find first/all unique elements”	Frequency + scan	HashMap count, then iterate original
“Find longest consecutive sequence”	Sequence start detection	HashSet, check num-1 absence

Decision Framework

Question	Answer
Need to associate data with each key?	HashMap
Need only existence or uniqueness?	HashSet
Need sorted order of keys?	TreeMap / TreeSet
Need insertion order preserved?	LinkedHashMap / LinkedHashSet
Fixed small key range (e.g., 26 letters)?	int[] array (faster than HashMap)

Key Java Idioms

Task	Code
Frequency counting	freq.put(key, freq.getOrDefault(key, 0) + 1)
Grouping	map.computeIfAbsent(key, k → new ArrayList<>()).add(item)
Duplicate check	if (!set.add(num)) { found duplicate }
Safe null handling	map.getOrDefault(key, defaultValue)

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina